



TALLER INTENSIVO

PRUEBAS UNITARIAS

Domina el arte de escribir código confiable con tests unitarios



Nivel: Principiante → Avanzado



Duración: 4 horas



CONTENIDO DEL CURSO



01

Fundamentos

Conceptos, pirámide de testing, frameworks y herramientas esenciales

02

TDD

Ciclo Red-Green-Refactor y metodología Test-Driven Development

03

Patrón AAA

Arrange-Act-Assert: estructura profesional para tus tests

04

Mocks y Stubs

Aislar dependencias con test doubles y simulaciones

05

Taller Práctico

3 ejercicios hands-on: String Calculator, Validador, Refactoring

06

Métricas y Calidad

Cobertura de código, mejores prácticas y checklist profesional



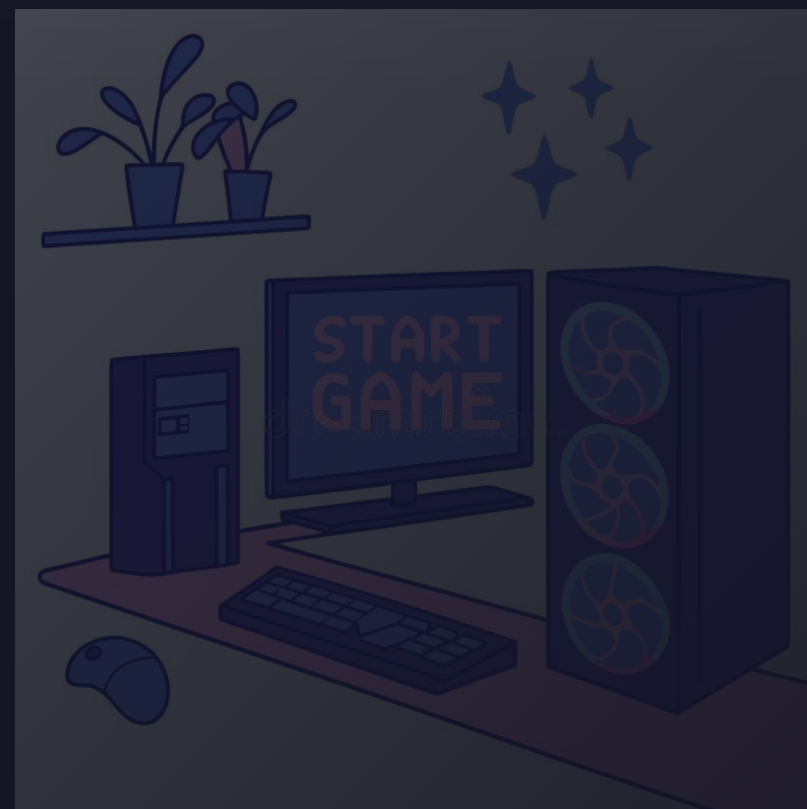
Objetivo: Al completar este taller, serás capaz de escribir tests unitarios efectivos, aplicar TDD, usar mocks y medir la calidad de tu código.



NIVEL 01

FUNDAMENTOS

Comprendiendo qué son las pruebas unitarias y por qué son esenciales para escribir software de calidad



¿QUÉ SON LAS PRUEBAS UNITARIAS?

Definición

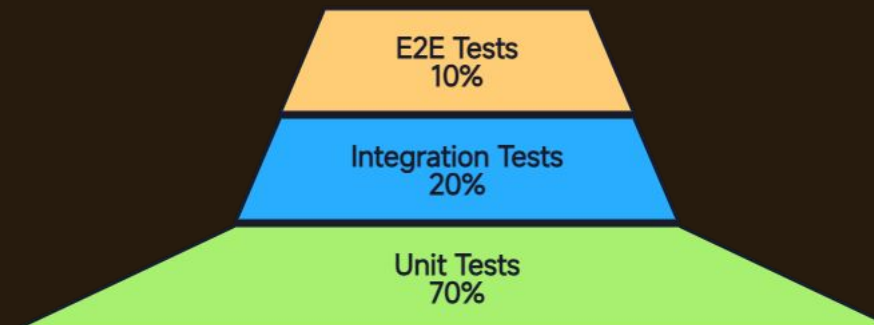
Procedimientos que verifican el funcionamiento de una **"unidad" de código** (método, función o clase) de forma **aislada e independiente**.

“ Garantizan que cada parte funcione correctamente **antes** de integrarse con otras partes del sistema.

Beneficios Clave

- ✓ **Detectar bugs temprano:** En etapas iniciales, reduciendo costos de corrección
- ✓ **Facilitar refactoring:** Modificar código sin miedo a romper funcionalidad
- ✓ **Documentación viva:** Los tests describen el comportamiento esperado
- ✓ **Reducir deuda técnica:** Código más limpio y mantenible

Pirámide de Testing



Características de Unit Tests

- ⚡ Rápidos (<10ms)
- 🛡️ Aislados
- 🔄 Repetibles
- ✅ Deterministas



FRAMEWORKS Y HERRAMIENTAS



Jest

JavaScript/TypeScript

- ★ Zero-config
- ★ Mocking integrado
- ★ Snapshot testing
- ★ Coverage built-in



JUnit 5

Java

- ★ Annotations
- ★ Parametrized tests
- ★ Extensions
- ★ IDE integration



pytest

Python

- ★ Simple y expresivo
- ★ Fixtures potentes
- ★ Plugins ecosystem
- ★ Parametrize



NUnit

.NET/C#

- ★ Attributes
- ★ Data-driven
- ★ Parallel execution
- ★ Fluent assertions

Comparativa Rápida

Característica	Jest	JUnit 5	pytest	NUnit
Configuración inicial	Zero-config	Moderada	Mínima	Moderada
Mocking integrado		(Mockito)	(unittest.mock)	
Ejecución paralela			(pytest-xdist)	
Curva de aprendizaje	Baja	Media	Baja	Media

💡 **Recomendación:** Elige el framework según tu stack tecnológico. Todos son excelentes opciones con comunidades activas y documentación completa.

NIVEL 02

TDD

Aprende a escribir tests antes que código con la metodología
Test-Driven Development



EL CICLO RED-GREEN-REFACTOR

01 RED

Escribe un test que falle. Define el comportamiento esperado antes de implementar.

🔧 Clarifica requisitos y establece la interfaz.

02 GREEN

Escribe código mínimo para pasar el test. No busques elegancia todavía.

✅ Objetivo: que el test pase lo antes posible.

03 REFACTO

Mejora el código manteniendo todos los tests verdes.

✏ Elimina duplicación, mejora nombres, simplifica.

Ejemplo Práctico: Calculadora

■ RED - Test que falla

```
// Test: sumar dos números test('sumar 2
+ 3 = 5', () => { expect(sumar(2,
3)).toBe(5); }); // ❌ FAIL: sumar is not
defined
```

■ GREEN - Mínimo código

```
// Implementación mínima function
sumar(a, b) { return 5; // Hardcode para
pasar } // ✅ PASS
```

■ REFACTOR - Mejora

```
// Nuevo test para forzar generalización
test('sumar 5 + 7 = 12', () => { expect(sumar(5,
7)).toBe(12); }); // ❌ FAIL → Refactor: function
sumar(a, b) { return a + b; // Solución real } // ✅
PASS - Todos los tests pasan
```

💡 **Tip:** Cada ciclo debe durar **2-5 minutos**. Si tardas más, el paso es demasiado grande.

⚠ **Regla de oro:** No escribas código nuevo a menos que un test esté fallando.

PATRÓN AAA: ARRANGE-ACT-ASSERT

Las 3 Fases

01 ARRANGE

Preparar precondiciones: crear objetos, configurar mocks, establecer datos de entrada.

02 ACT

Ejecutar el método a probar. Idealmente **una sola línea**.

03 ASSERT

Verificar resultados: valores retornados, estado de objetos, llamadas a mocks.

Ventajas del Patrón

- ✓ **Legibilidad:** Tests se leen como especificaciones
- ✓ **Mantenibilidad:** Fácil de entender y modificar
- ✓ **Debugging:** Identificar fallas rápidamente

Ejemplo: JavaScript/Jest

```
test('autenticar usuario válido', () => {  
  const mockUserService = { findUser:  
    jest.fn().mockReturnValue({ id: 1, name: 'Ana' })  
  };  
  const authService = new AuthService(mockUserService);  
  const email = 'ana@example.com'; const password =  
    'correcta';  
  const result = authService.login(email, password);  
  expect(result.isAuthenticated).toBe(true);  
  expect(result.user.name).toEqual('Ana');  
  expect(mockUserService.findUser).toHaveBeenCalledWith(em  
    ail);  
});
```

Ejemplo: Python/pytest

```
def test_aplicar_cupon_valido(): # ARRANGE cart = Cart(total_price=100.00) cupon = "SAVE10" # ACT cart.apply_coupon(cupon) # ASSERT assert cart.total_price == 90.00  
def test_password_rechazada_si_muy_corta(): #  
  ARRANGE password_corta = "pw" # ACT es_valida = is_valid_password(password_corta) # ASSERT assert es_valida is False
```

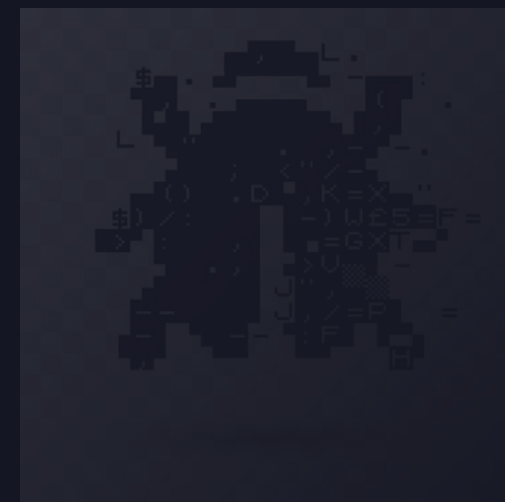
★ **Buena práctica:** Deja una **línea en blanco** entre cada sección (Arrange, Act, Assert) para mejorar legibilidad.



NIVEL 03

AISLANDO DEPENDENCIAS

Técnicas avanzadas: mocks, stubs y test doubles para tests verdaderamente aislados



MOCKS Y STUBS EN ACCIÓN

Stubs vs Mocks

STUBS

Proveen **respuestas predefinidas** a llamadas durante el test.

→ Simulan dependencias externas (DB, APIs)

MOCKS

Verifican **interacciones**: qué métodos fueron llamados, con qué argumentos.

→ Aseguran que el código interactúa correctamente

¿Cuándo usar cada uno?

- ✓ **Stub**: Cuando necesitas datos de una dependencia
- ✓ **Mock**: Cuando necesitas verificar comportamiento
- ✓ **Ambos**: Mock que también actúa como stub

Ejemplo Práctico

Servicio **UserService** que depende de **ApiClient**:

```
class UserService {  
  constructor(apiClient) {  
    this.apiClient = apiClient;  
  }  
  async getUsername(userId)  
  { try { const userData = await this.apiClient.getUserData(userId); return  
    userData.name;  
  }  
  catch (error)  
  { return "User not found"; }  
}
```

```
import { jest } from '@jest/globals';  
test('maneja error de API', async () => {  
  const mockApi = { getUserData: jest.fn().mockRejectedValue(new Error("API caída")) }; const  
  service = new UserService(mockApi);  
  const result = await service.getUsername(123);  
  expect(result).toBe("User not found"); expect(mockApi.getUserData).toHaveBeenCalledTimes(1);  
});
```

❗ **Regla importante:** Mock **roles**, no **objetos**. Enfócate en dependencias que cruzan límites arquitectónicos (DB, red, filesystem).

NIVEL 04

TALLER PRÁCTICO

Manos al código: 3 ejercicios hands-on para aplicar todo lo aprendido



STRING CALCULATOR CON TDD

Descripción

Implementa **add(string)** que recibe números separados por delimitadores y retorna la suma.

`</>` Aplica TDD: escribe test → implementa → refactoriza.

Pasos Incrementales

- 1 **String vacío** → retorna 0
- 2 **Un número** → retorna ese número
- 3 **Dos números** → retorna la suma
- 4 **N números** → maneja cantidad variable
- 5 **Saltos de línea** → '\n' como separador
- 6 **Números negativos** → lanza excepción

Tests Iniciales

```
describe('String Calculator', () => { test('string vacío retorna 0',  
  () => { expect(add('')).toBe(0); }); test('un número retorna el  
  número', () => { expect(add('5')).toBe(5); }); test('dos números  
  retorna suma', () => { expect(add('1,2')).toBe(3); }); test('N  
  números retorna suma', () => { expect(add('1,2,3,4,5')).toBe(15);  
  }); test('maneja saltos de línea', () => {  
    expect(add('1\n2,3')).toBe(6); }); test('números negativos lanzan  
  error', () => { expect(() => add('1,-2,3')).toThrow('Negativos no  
  permitidos: -2'); }); });
```

Pistas

- 💡 Usa **split()** para separar números
- 💡 Convierte strings a números con **parseInt()**
- 💡 Usa **reduce()** para sumar el array

VALIDADOR DE CONTRASEÑAS

Descripción

Implementa `isValidPassword(password)` que valide según reglas de seguridad.

🛡️ Aplica **tests parametrizados** y **SRP**.

Reglas de Validación

- ✔️ Mínimo **8 caracteres**
- ✔️ Al menos **una mayúscula**
- ✔️ Al menos **una minúscula**
- ✔️ Al menos **un número**
- ✔️ Al menos **un carácter especial** (!@#\$%^&*)

Tests Parametrizados

```
import pytest @pytest.mark.parametrize("password, expected", [ # Casos válidos ("Password1!", True), ("Secure@123", True), ("MyP@ssw0rd", True), # Casos inválidos ("abc", False), # muy corta ("password", False), # sin mayúscula ("PASSWORD1", False), # sin minúscula ("Password!", False), # sin número ("Password1", False), # sin especial ("", False), # vacía ]) def test_password_validation(password, expected): assert is_valid_password(password) == expected # Tests individuales para cada regla def test_password_rechazada_si_muy_corta(): assert is_valid_password("abc") is False def test_password_rechazada_sin_mayuscula(): assert is_valid_password("password1!") is False def test_password_rechazada_sin_minuscula(): assert is_valid_password("PASSWORD1!") is False
```

Casos de Prueba

VÁLIDOS ✔️	INVÁLIDOS ❌
"Password1!"	"abc" (corta)
"Secure@123"	"password" (sin mayús.)
"MyP@ssw0rd"	"PASSWORD1" (sin minús.)
	"Password!" (sin número)

Pistas

- 💡 Usa **expresiones regulares** para validar
- 💡 Un test por cada **regla individual**
- 💡 Testea **casos límite**: 7 chars, 8 chars, 9 chars

REFACTORING CON TESTS DE SEGURIDAD

Descripción

Tienes código **legacy** de una tienda que actualiza calidad de items. Debes:

- 1 Escribir **tests de caracterización**
- 2 **Refactorizar** aplicando clean code
- 3 Asegurar que **todos los tests pasan**

Código Legacy

```
class GildedRose { constructor(items) { this.items = items; } updateQuality() { for (let i = 0; i < this.items.length; i++) { if (this.items[i].name !== 'Aged Brie' && this.items[i].name !== 'Backstage passes') { if (this.items[i].quality > 0) { if (this.items[i].name !== 'Sulfuras') { this.items[i].quality--; } } } else { if (this.items[i].quality < 50) { this.items[i].quality++; if (this.items[i].name === 'Backstage passes') { if (this.items[i].sellIn < 11) { if (this.items[i].quality < 50) { this.items[i].quality++; } } // ... más código spaghetti } } } } } } // ... (código continúa) } }
```

Tests de Caracterización

```
describe('GildedRose', () => { test('item normal: calidad baja', () => { const items = [new Item('Normal', 5, 10)]; const gildedRose = new GildedRose(items); gildedRose.updateQuality(); expect(items[0].quality).toBe(9); expect(items[0].sellIn).toBe(4); }); test('Aged Brie: calidad aumenta', () => { const items = [new Item('Aged Brie', 5, 10)]; const gildedRose = new GildedRose(items); gildedRose.updateQuality(); expect(items[0].quality).toBe(11); }); test('Sulfuras: no cambia', () => { const items = [new Item('Sulfuras', 5, 80)]; const gildedRose = new GildedRose(items); gildedRose.updateQuality(); expect(items[0].quality).toBe(80); expect(items[0].sellIn).toBe(5); }); test('Backstage: quality +2 si sellIn <= 10', () => { const items = [new Item('Backstage passes', 10, 10)]; const gildedRose = new GildedRose(items); gildedRose.updateQuality(); expect(items[0].quality).toBe(12); }); });
```

Checklist de Refactoring

- Extraer métodos con nombres descriptivos
- Eliminar código duplicado
- Usar **switch** o **strategy pattern**
- Ejecutar tests después de cada cambio



NIVEL 05

MÉTRICAS Y CALIDAD

Midiendo la efectividad de tus pruebas y aplicando mejores prácticas profesionales



COBERTURA DE CÓDIGO Y MÉTRICAS CLAVE

¿Qué mide la cobertura?

Porcentaje de código ejecutado durante los tests. Mide **líneas**, **branches**, **funciones** y **statements**.

Líneas



Líneas de código ejecutadas

Branches



Ramas if/else ejecutadas

Funciones



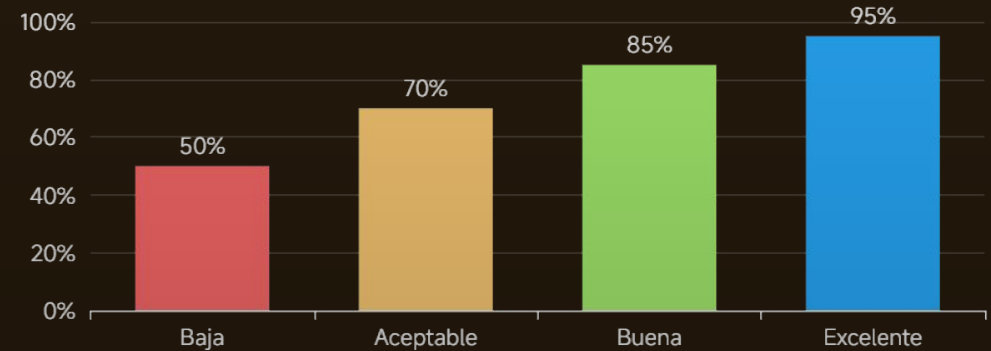
Métodos llamados

Statements



Sentencias ejecutadas

Objetivos Realistas



Herramientas por Lenguaje



JavaScript: Istanbul/nyc



Java: JaCoCo



Python: Coverage.py



.NET: Coverlet

Buenas Prácticas

- ✓ Meta: **70-85%** (no 100%)
- ✓ Analiza **reportes detallados**
- ✓ Integra en **CI/CD** como quality gate
- ✓ Monitorea **tendencias** en el tiempo

Anti-patrones

- ✗ Perseguir **100% cobertura** sin calidad
- ✗ Escribir tests solo para **subir métrica**
- ✗ Ignorar **branches no cubiertos**

MEJORES PRÁCTICAS Y CHECKLIST

Principios Fundamentales

- 01 Un test, una responsabilidad**
Cada test verifica un solo comportamiento
- 02 Nombres descriptivos**
`shouldReturnErrorWhenPasswordIsTooShort`
- 03 Tests independientes**
Sin dependencias, pueden ejecutarse en paralelo
- 04 Usar fixtures y builders**
Setup reutilizable, reduce duplicación
- 05 Testear casos de borde**
Null, empty, valores límite, errores

Prácticas Avanzadas

- 06 Evitar lógica condicional**
No uses `if/else` en tests
- 07 Tests rápidos**
Ejecución < 10ms por test
- 08 CI/CD en cada commit**
Automatizar ejecución continua
- 09 Refactorizar tests**
Mantén tests limpios y actualizados
- 10 Tests como documentación**
Deben ser legibles por humanos

Métricas de Calidad

Tiempo de ejecución	< 10ms
Cobertura objetivo	70-85%
Tests por función	3-5 mínimo
Ratio test/código	1:1 - 2:1

Checklist Final

- ☐ Tests siguen patrón AAA
- ☐ Nombres descriptivos y claros
- ☐ Un assert por test (idealmente)
- ☐ Mocks para dependencias externas
- ☐ Casos de borde cubiertos
- ☐ Cobertura > 70%
- ☐ Tests ejecutan en CI/CD

NIVEL
COMPLETADO

¡FELICIDADES!

Has completado el taller de Pruebas Unitarias. Ahora tienes las herramientas para escribir código confiable y mantenible.

Recursos Recomendados



Libros:

"TDD" - Kent Beck

"Clean Code" - Robert C. Martin



Katas de TDD:

String Calculator, Bowling Game, FizzBuzz, Gilded Rose



Plataformas:

Cyber-Dojo, Codewars, Coding Dojo

Próximos Pasos

1

Implementa tests en tu **proyecto actual**

2

Configura **CI/CD** con tests automatizados

3

Comparte conocimiento con tu **equipo**

4

Practica **katas semanalmente**



"Un código sin tests es código roto esperando a fallar. Ahora tienes el poder de prevenirlo."

ACTIVIDAD

¡PRUEBAS UNITARIAS!

Tomar como referencia el anterior taller para realizar una presentación de mínimo 20 minutos.

Seleccionar un lenguaje de programación

PHP
JAVA
NODE.JS
PYTHON
JAVASCRIPT

GRUPOS DEL PROYECTO

- 1 Explicar la importancia de las pruebas unitarias.
- 2 Realizar un ejemplo del proceso, según el lenguaje de programación.
- 3 Crear una plantilla de plan de pruebas unitarias.
- 4 Formular 5 preguntas de la presentación.

FECHA DE ENTREGA Y EXPOSICIÓN: 23/05/2026